

Real-time Grasping Pipeline for Mobile Manipulators via Language-Grounded Perception and Reactive Control

Hardware and System Objective

본 연구는 **Jetson AGX Orin 64GB**를 탑재한 **Mobile Manipulator** 환경에서 전체 파이프라인이 실시간(최소 5FPS, 목표 10FPS 이상)으로 동작하는 것을 목표로 합니다. 로봇 팔은 **6-DOF Manipulator Arm**을 사용합니다.

1. Key Idea

본 연구의 핵심 아이디어는 Edge Device 환경에서 Natural Language Command 입력부터 로봇의 Grasping 동작에 이르는 전체 과정을 실시간으로 수행하는 통합 파이프라인을 구축하는 것입니다. 사용자가 자연어 명령을 내리면, 시스템은 **(1) Perception Module**을 통해 타겟 객체를 인식 및 분할하고, **(2) Decision Module**을 통해 강건한 파지점을 결정하며, **(3) Control Module**을 통해 **Gaze Condition**을 만족하는 부드러운 궤적을 생성하여 객체를 파지합니다. 특히, 복잡한 Grasping Pose Prediction 대신 PCA 기반의 기하학적 분석을 통해 파지점을 결정함으로써 전체 시스템의 경량화와 실시간성을 확보합니다.

2. Perception Module

Perception Module은 Natural Language Command를 입력받아 타겟 객체를 정확하게 인지하고, Segmentation Mask를 출력하는 역할을 수행합니다. 본 모듈은 **NanoOWL**에서 영감을 받아 **OWL-ViT**와 **NanoSAM**을 결합한 구조로 설계되었습니다. 기존 **NanoOWL**이 명사구만 인식 가능한 한계를 극복하기 위해, **Image Encoder**와 **Text Encoder**를 분리하여 문장 전체의 이해가 가능하도록 확장했습니다.

전체 파이프라인은 4개의 독립적인 TensorRT Engine으로 구성됩니다:

- OWL-ViT Text Encoder**: 자연어 명령을 임베딩 벡터로 변환 (명령 변경 시에만 1회 실행 후 캐싱)
- OWL-ViT Image Encoder**: 입력 이미지에서 시각적 특징 추출
- NanoSAM Image Encoder**: 입력 이미지에서 분할을 위한 특징 추출
- NanoSAM Image Decoder**: 두 Encoder의 특징과 텍스트 임베딩을 종합하여 최종 Segmentation Mask 생성

매 프레임마다 동일한 이미지를 입력받는 OWL-ViT와 NanoSAM의 Image Encoder는 병렬로 실행하여 처리 속도를 극대화합니다. 또한, CUDA Graph, 비동기(Asynchronous) 실행, JIT(Just-In-Time) 컴파일 등 GPU 최적화 기법을 총동원하여 CPU-GPU 간 데이터 전송 오버헤드를 최소화하고 극한의 추론 성능을 달성했습니다.

3. Decision Module

Decision Module은 Perception Module에서 출력된 Segmentation Mask를 입력받아 최종 End-Effector 목표 지점(Target Point)을 결정합니다. 본 연구에서는 높은 계산 비용과 외부 라이브러리 의존성을 갖는 Grasping Pose Prediction 알고리즘을 배제하고, 기하학적 분석에 기반한 강건하고 효율적인 방법을 제안합니다.

처리 과정은 다음과 같습니다:

- Partial Point Cloud Generation**: Segmentation Mask에 해당하는 영역의 Depth 정보를 활용하여 3D Partial Point Cloud를 생성합니다.

2. **Initial Center and Axes via PCA:** 생성된 Point Cloud에 주성분 분석(Principal Component Analysis, PCA)을 적용하여 3개의 직교 주축 벡터 $\{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3\}$ (고유값 $\lambda_1 \geq \lambda_2 \geq \lambda_3$)와 초기 기하학적 중심(Geometric Centroid)을 얻습니다.
3. **Centroid Refinement (1st Stage):** 초기 중심점은 Partial Point Cloud의 편향(bias)으로 인해 실제 객체의 중심과 다릅니다. 이를 보정하기 위해 모든 점을 주축 $\mathbf{v}_1$ 에 투영(projection)한 후, 투영된 1차원 데이터의 최솟값과 최댓값의 중간 지점을 찾아 **1차 보정 중심점**을 계산합니다.
4. **Centroid Refinement (2nd Stage):** 1차 보정 중심점을 지나고 $\mathbf{v}_3$ 를 법선 벡터로 갖는 대칭 기준면(Symmetry Plane)을 설정합니다. 원본 Point Cloud를 이 평면에 대해 반사(reflection)시켜 가상의(virtual) Point Cloud를 생성합니다. 최종적으로 원본과 가상 Point Cloud를 모두 포함한 전체 집합의 산술 평균을 계산하여 **최종 중심점**을 결정합니다.

이 최종 중심점이 End-Effector Tool Frame이 도달해야 할 Target Point가 됩니다.

4. Control Module

Control Module은 Decision Module에서 결정된 Target End-Effector Pose를 입력받아, 충돌 없이 **Gaze Condition**을 항상 만족하며 목표 지점까지 이동하는 반응형(Reactive) 제어를 수행합니다. 본 모듈의 핵심은 **실시간 반응성(Real-time Reactivity)**, **궤적의 부드러움(Smoothness of Trajectory)**, 그리고 ****Gaze Condition의 강건성(Robustness of Gaze Condition)****을 모두 만족시키는 계층적 제어 아키텍처(Hierarchical Control Architecture)를 설계하는 것입니다. 제어 문제는 ****고정된 타겟 객체(Fixed Target Object)****를 가정하고 진행합니다.

Gaze Condition이란, 실시간 Perception-Decision 파이프라인을 연속적으로 실행하기 위해 로봇의 End-Effector에 장착된 카메라가 어떤 위치와 경로로 움직이든 항상 타겟 객체를 시야에 두어야 한다는 제약 조건입니다.

4.1. Offline Phase: Gaze Condition Reachability Map

Offline 단계의 핵심은 실시간 제어의 부담을 줄이기 위해 **Gaze Condition**을 Workspace 자체에 내재화하는 **Gaze Condition Reachability Map**을 생성하는 것입니다. Manipulator의 작업 공간(Workspace)을 Voxel Grid로 분할하고, 각 Voxel에 대해 End-Effector의 도달 가능 여부를 계산합니다. 이때, 단순히 도달 가능성(Binary)만 저장하는 것이 아니라, 각 Voxel의 위치(x, y, z)에서 객체를 바라보는 고정된 방향(Fixed Orientation: ϕ, θ, ψ)을 만족하는 Joint Configuration이 존재하는지를 C-Space 상의 충돌(Self-collision, Environment collision)까지 고려하여 계산합니다. 이 과정을 통해 Online 단계에서 경로 계획 시 **Gaze Condition**과 충돌 회피를 자명하게 만족하는 Free Voxel 집합을 미리 확보할 수 있습니다.

4.2. Online Phase: Real-time Reactive Control

Online 단계는 Perception 및 Decision Module의 출력을 받아 실시간으로 로봇을 제어하는 핵심 부분이며, 총 3단계의 프로세스로 구성됩니다.

4.2.1. Global Path Planning

Offline에서 생성된 Reachability Map 상에서 현재 End-Effector Pose에서 목표 Voxel까지의 경로를 A* 또는 RRT*와 같은 탐색 기반 알고리즘을 사용하여 계획합니다. 이 과정의 결과물은 **Gaze Condition**과 C-Space 충돌 회피가 보장된 이산적인 Waypoint 시퀀스(Discrete Waypoint Sequence)입니다.

4.2.2. Smooth Trajectory Interpolation

탐색 기반으로 생성된 이산적인 Waypoint들은 로봇의 물리적 한계를 고려하지 않아 Jerk가 발생할 수 있습니다. 이를 해결하기 위해 Waypoint들을 **Quintic Spline Interpolation**을 사용하여 시간에 대한 연속적인 궤적 ($\mathbf{x}_d(t)$)으로 변환합니다. 이 궤적은 위치, 속도, 가속도가 모두 연속인 $\mathbf{C}^2$ 연속성 (Continuity)을 가지므로, 물리적으로 자연스럽고 Jerk를 최소화하는 매우 부드러운 움직임을 생성합니다. 이 단계의 최종 출력은 실시간 제어기에 필요한 목표 위치 $\mathbf{x}_d(t)$ 와 목표 속도 $\dot{\mathbf{x}}_d(t)$ 프로파일입니다.

4.2.3. Real-Time Reactive Control via DIK

보간된 궤적을 강건하게 추종하기 위해 높은 주파수(e.g., 100Hz 이상)로 동작하는 **Differential Inverse Kinematics (DIK)** 기반의 Closed-loop Controller를 사용합니다. 제어 법칙은 다음과 같습니다:

$$\dot{\mathbf{q}} = J^{\dagger}(\mathbf{q}) (\dot{\mathbf{x}}_d(t) + K_p(\mathbf{x}_d(t) - \mathbf{x}_{\text{current}}))$$

- $\dot{\mathbf{x}}_d(t)$: 보간된 궤적으로부터 제공되는 **Feedforward 제어** 항입니다. 이 항은 시스템이 목표 궤적을 지체 없이 따라가도록 유도하여 추종 성능을 극대화합니다.
- $K_p(\mathbf{x}_d(t) - \mathbf{x}_{\text{current}})$: 현재 상태와 목표 사이의 오차를 보상하는 **Feedback 제어** 항입니다. 모델링의 불확실성이나 외부 외란(External Disturbances)에 대한 강건성을 확보합니다.

이러한 속도 기반 제어 방식은 매 순간의 계산량이 매우 적어 실시간성을 보장하며, Perception 및 Decision 모듈의 결과가 동적으로 변하더라도 즉각적으로 반응하여 궤적을 수정할 수 있는 높은 반응성(Reactivity)을 제공합니다.

결론적으로, 본 연구에서 제안하는 계층적 제어 구조는 복잡한 제약 조건 하에서도 **안정성(Stability)**, **반응성(Reactivity)**, 그리고 **움직임의 품질(Quality of Movement)**을 모두 확보하는 강건한 솔루션을 제공하는 것을 목표로 합니다.